

STAT 4220
Wren Nicol Final Project

For my final project, I decided to do a full factorial design 2^k , with three factors. Factor A is the arm position, and I chose the $-$ to be the lower position, which is position 2, and the $+$ position to be the top position, or position 1. For Factor B, I chose this to be the vertical position and I chose $-$ to be position 4, and $+$ to be position 1. Finally, for my last factor, I chose the stop angle, with $+$ being 120° and $-$ 90° . The reason I chose this design is because it would efficiently look at all of the factors while minimizing the amount of runs needed. It also allows for all of the interaction effects to be tested for significance. Below is the table of the factors.

Table 1: Factors and Levels, Catapult Experiment

Factor	Level	
	$-$	$+$
A. Arm Position	2	1
B. Vertical Position	4	1
C. Stop Angle ($^\circ$)	90	120

I decided to ignore Factor 1, which was the type of ball, and Factor 5, which was the height. I did this because Factor 5 was already fixed, and because I didn't believe that the type of ball would affect the results, and I was more interested in the actual functions of the catapult. I also decided to do three replicates for each run, so I ended up having 24 observations. I used a random number generator to randomize the order of the runs. I used a tape measure to collect the data of how far the ball went, and measured in inches.

Table 2: Planning Matrix

Run Order	Arm Position	Vertical Position	Stop Angle	Randomized Run
1	$-$	$-$	$-$	5, 8, 20
2	$-$	$-$	$+$	2, 9, 14
3	$-$	$+$	$-$	6, 11, 15
4	$-$	$+$	$+$	16, 17, 22
5	$+$	$-$	$-$	3, 12, 21
6	$+$	$-$	$+$	4, 7, 23
7	$+$	$+$	$-$	10, 18, 24
8	$+$	$+$	$+$	1, 13, 19

I then collected my data, and one thing I found out from collecting the data was that Factor A, the arm position, was quite costly to change for each run, since it took a while to screw and unscrew the nail. If I were redoing this project, I would have done a restricted randomization

on Factor A, so that it would have gone quicker. This is a limitation to my design, because this decreases the efficiency of collecting data and makes it time costly.

Table 3: Data Collection

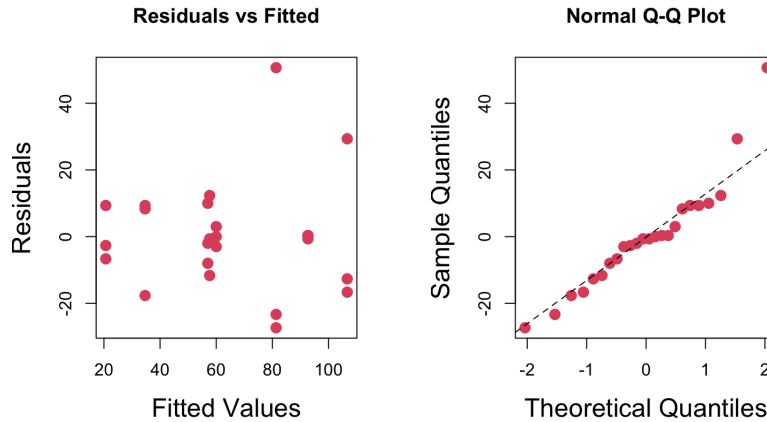
Arm Position	Vertical Position	Stop Angle	Distance (in.)
—	—	—	14, 18, 30
—	—	+	17, 43, 44
—	+	—	46, 57, 70
—	+	+	57, 60, 63
+	—	—	54, 58, 132
+	—	+	49, 55, 67
+	+	—	92, 93, 93
+	+	+	90, 94, 136

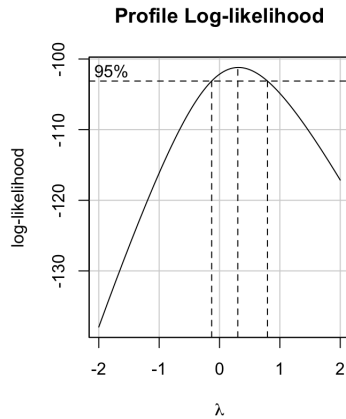
The model for my experiment is:

$$y_{ijkl} = \mu + \alpha_i + \beta_j + \gamma_k + (\alpha\beta)_{ij} + (\alpha\gamma)_{ik} + (\beta\gamma)_{jk} + (\alpha\beta\gamma)_{ijk} + \varepsilon_{ijkl} \quad (1)$$

Where μ is the overall mean, α_i is the main effect of Factor A (arm position), β_j is the main effect of Factor B (vertical position), and γ_k is the main effect of Factor C (stop angle). $(\alpha\beta)_{ij}$ is the two way interaction of A and B, $(\alpha\gamma)_{ik}$ is the two way interaction of A and C, $(\beta\gamma)_{jk}$ is the two way interaction of B and C, and $(\alpha\beta\gamma)_{ijk}$ is the three way interaction of A, B, and C. ε_{ijkl} is the error term that is $N(0, \sigma^2)$. i, j, k are $(+, -)$, and $l = 1, 2, 3$.

Since in my data collection I found there to be two large values (132, 136), I decided to check for my assumptions first. Below is the Q-Q plot and the residual plot, where we can see that the two outliers have an effect on the normality and homoscedasticity conditions. There is some slight fanning in the residual plot, and the right tail on the Q-Q plot is skewed. Because of this, I decided to look for a BoxCox transformation before proceeding with my analysis.

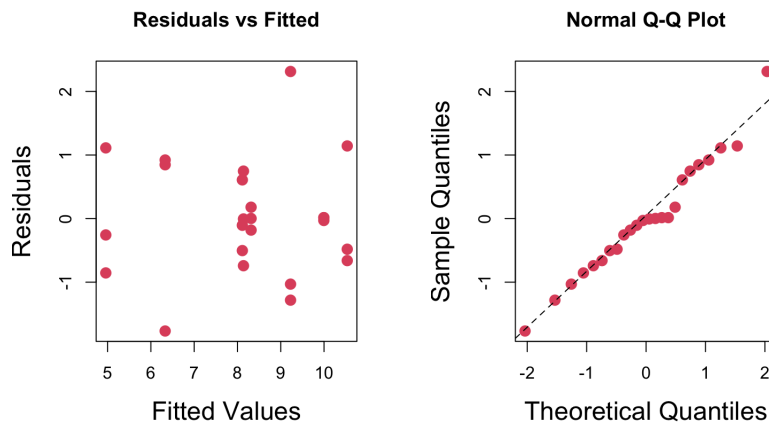




BoxCox Plot

Using the `powerTransform` function, I found that the optimal transformation parameter is $\lambda = 0.3131548$. I then applied this transformation to the response variable and fit the new model. After fitting the new model, I checked the assumptions again by finding the new residual and Q-Q plot below. As we can see, the normality and homoscedasticity condition is now satisfied, and we can continue with finding the effects. After applying the Box-Cox transformation, the influence of the extreme observations (132, 136) was reduced, and model assumptions were satisfied.

$$y^{(0.313)} = \frac{y^{0.313} - 1}{0.313} \quad (2)$$



I found the coefficients of the model to be:

```
> g2$coef
      (Intercept)      arm_pos      vert_pos
      8.20152733      1.26597838      1.04467000
      stop_ang      arm_pos:vert_pos      arm_pos:stop_ang
```

0.12176024	-0.24664825	-0.26489913
vert_pos:stop_ang	arm_pos:vert_pos:stop_ang	
0.05737943	0.35622151	

Now, I used these coefficients to get the fitted model equation to be:

$$\hat{y}^{(0.313)} = 8.202 + 1.266x_A + 1.045x_B + 0.122x_C - 0.247x_Ax_B - 0.265x_Ax_C + 0.057x_Bx_C + 0.356x_Ax_Bx_C \quad (3)$$

Looking at the ANOVA table below, it is clear that the only significant factors are the arm position and the vertical position, and that stop angle and all interactions are insignificant. We will use a half normal plot to visualize the significant vs. insignificant factors. Now, we can also think about creating a reduced model based on this, for simplicity and reducing $\hat{\sigma}$.

Table 4: ANOVA Table, Full Model

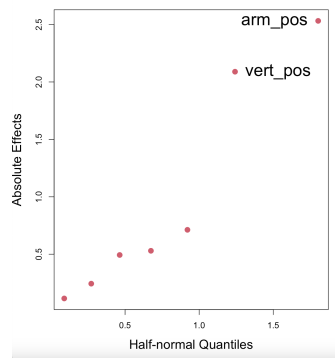
Source	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Arm Position	1	38.46	38.46	33.133	2.95e-05
Vert. Position	1	26.19	26.19	22.561	0.000217
Stop Angle	1	0.36	0.36	0.306	0.587497
Arm:Vert	1	1.46	1.46	1.258	0.278647
Arm:Stop	1	1.68	1.68	1.451	0.245938
Vert:Stop	1	0.08	0.08	0.068	0.797506
Arm:Vert:Stop	1	3.05	3.05	2.623	0.124847
Residuals	16	18.58	1.16		
Total	23	89.86			

Then, to find which variables are significant, we must multiply these coefficients by 2 to eventually use for the half normal plot.

```
> eff2
      arm_pos      vert_pos      stop_ang
      2.5319568      2.0893400      0.2435205
arm_pos:vert_pos arm_pos:stop_ang vert_pos:stop_ang
      -0.4932965      -0.5297983      0.1147589
arm_pos:vert_pos:stop_ang
      0.7124430
```

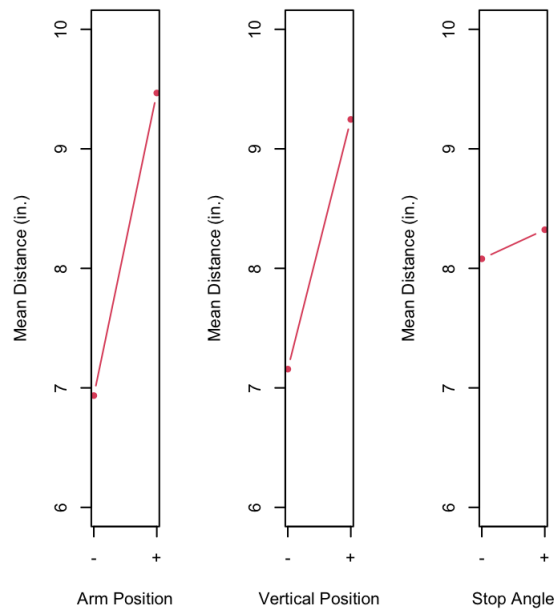
From these coefficients, we interpret the effects on the transformed response variable. The two factors with the largest effects are arm position and vertical position. Increasing arm position to the (+) level increases the transformed response by approximately 2.53 units, while increasing vertical position increases it by about 2.09 units. This indicates these factors have the strongest influence on the response after the transformation.

Then I created the half normal plot in order to see the effects that were significant. In the plot below, we can see that the significant effects were Factor A, the arm position, and Factor B, the vertical position.



Half Normal Plot

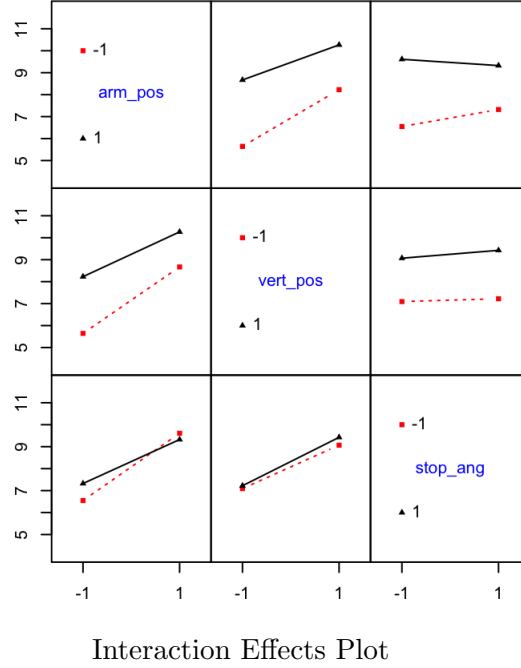
The main effects plot is provided below to corroborate what the half normal plot showed, which is that factor A and B have very steep slopes, meaning they are significant, whereas C has a small slope, showing insignificance.



Main Effects Plot

The interaction plot below also confirms what was shown in the half normal plot, since it shows approximately parallel lines across all two way interactions, meaning that there are no significant two way interactions.

Interaction plot matrix for y_transforr



Since the only two effects that were significant were the vertical position and the arm position, the next logical thing to do would be to fit a reduced model with only these variables, with all of the other variables included in the residual term. The final equation shows a cleaner model without noise, fitting only the significant effects.

$$\hat{y}^{(0.313)} = 8.20 + 1.266x_A + 1.045x_B \quad (4)$$

Table 5: ANOVA Table, Catapult Experiment

Source	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Arm Position	1	38.46	38.46	32.05	1.28e-05
Vert. Position	1	26.19	26.19	21.83	0.00013
Residuals	21	25.20	1.20		
Total	23	89.85			

The reduced model has an R^2 value of 0.7196, and a $\hat{\sigma}$ value of 1.095. The original model with all terms had an R^2 of 0.7933, and a $\hat{\sigma}$ value of 1.077. We use the reduced model because it includes only the statistically significant effects, making it more interpretable and less prone to overfitting. Even though the original model has better R^2 and $\hat{\sigma}$ values, we should still fit the reduced model because the improvements in the original model come from added complexity.

Conclusion

A full 2^3 factorial model was initially fitted to the response variable, distance (inches), using arm position, vertical position, and stop angle, including all interaction terms. Plots of the residuals and the Q-Q plot indicated violation of the normality and homoscedasticity conditions, so a BoxCox transformation was applied to improve model assumptions. After transforming the response using $\lambda = 0.313$, the full model was refit and plots showing residuals appeared more randomly scattered and closer to normality on the Q-Q plot. To identify the most important effects, a half normal plot of the estimated factorial effects was used. This suggested that the main effects of arm position and vertical position were the most influential, while stop angle and interaction terms did not have much effect on the response. Based on this, a reduced model including only arm position and vertical position was fit. The reduced model provided a simpler explanation of the response with minimal loss of fit compared to the full model, making it easier for interpretation. Overall, this suggests that the main factors of variation in y are arm position and vertical position, while stop angle and interaction effects have little effect on the response.



(a) Catapult



(b) Factor A: Arm Position



(c) Factor B: Vertical Position



(d) Factor C: Stop Angle

Appendix - R Codes

```
#####  
#                                                                 #  
#           First we enter the data                             #  
#                                                                 #  
#####  
arm_pos = c(-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,1,1,1,1,1,1,1,1,1,1)  
vert_pos = c(-1,-1,-1,-1,-1,-1,1,1,1,1,1,1,-1,-1,-1,-1,-1,1,1,1,1,1)  
stop_ang = c(-1,-1,-1,1,1,1,-1,-1,-1,1,1,1,-1,-1,-1,1,1,1,-1,-1,1,1)  
y = c(14,18,30,17,44,43,70,46,57,63,57,60,132,54,58,55,67,49,93,92,93,94,90,136)  
cbind(arm_pos,vert_pos,stop_ang,y)  
#####  
#                                                                 #  
#           Now we fit the model                                 #  
#                                                                 #  
#####  
g = lm(y ~ arm_pos + vert_pos + stop_ang + arm_pos:vert_pos +  
        arm_pos:stop_ang + vert_pos:stop_ang + arm_pos:vert_pos:stop_ang)  
#####  
#                                                                 #  
#           Residual vs Fitted Plot                             #  
#                                                                 #  
#####  
plot(g$fitted.values, g$residuals,  
     xlab = "Fitted Values", ylab = "Residuals",  
     main = "Residuals vs Fitted",  
     pch = 20, cex = 2, col = 2, cex.lab = 1.5)  
#####  
#                                                                 #  
#           QQ Plot                                             #  
#                                                                 #  
#####  
qqnorm(g$residuals, pch = 20, cex = 2, col = 2, cex.lab = 1.5)  
qqline(g$residuals, lty = 2)  
#####  
#                                                                 #  
#           BoxCox Transformation                               #  
#                                                                 #  
#####  
library(car)  
boxCox(g)  
powerTransform(g)  
lambda = 0.3131548  
y_transformed = (y^lambda - 1) / lambda  
g2 = lm(y_transformed ~ arm_pos + vert_pos + stop_ang +
```

```

        arm_pos:vert_pos + arm_pos:stop_ang +
        vert_pos:stop_ang + arm_pos:vert_pos:stop_ang)
#####
#                                                                    #
#          Residual vs Fitted Plot                                #
#                                                                    #
#####
plot(g2$fitted.values, g2$residuals,
     xlab = "Fitted Values", ylab = "Residuals",
     main = "Residuals vs Fitted",
     pch = 20, cex = 2, col = 2, cex.lab = 1.5)
#####
#                                                                    #
#          QQ Plot                                              #
#                                                                    #
#####
qqnorm(g2$residuals, pch = 20, cex = 2, col = 2, cex.lab = 1.5)
qqline(g2$residuals, lty = 2)
#####
#                                                                    #
#          Coefficients                                         #
#                                                                    #
#####
g2$coef
#####
#                                                                    #
#          ANOVA Table                                          #
#                                                                    #
#####
summary.aov(g2)
#####
#                                                                    #
#          Factorial Effects                                    #
#                                                                    #
#####
eff2 = 2 * g2$coef[-1]
eff2
#####
#                                                                    #
#          Create half normal plot function                    #
#                                                                    #
#####
halfnorm = function(x)
{
  n = length(x)
  halfn = .5+.5*(1:n-.5)/n

```

```

plot(qnorm(halfn), sort(abs(x)), type="p", cex=2, pch=20,
     xlab="Half-normal Quantiles", ylab="Absolute Effects", cex.lab=1.5, col=2)
identify(qnorm(halfn), sort(abs(x)), names(sort(abs(x))), cex=2)
}

#####
#
#           Plot half normal for eff2
#
#####
halfnorm(eff2)
#####
#
#           Main Effects Plots
#
#####
par(mfrow = c(1,3))

factors = list(arm_pos, vert_pos, stop_ang)
names = c("Arm Position", "Vertical Position", "Stop Angle")
ylims = c(6, 10)

for(i in 1:3){
  means = c(mean(y_transformed[factors[[i]] == -1]), mean(y_transformed[factors[[i]] == 1]))
  plot(c(-1,1), means,
       type = "b", pch = 20, col = 2,
       xlab = names[i], ylab = "Mean Distance (in.)",
       ylim = ylims, xaxt = "n")
  axis(1, at = c(-1,1), labels = c("-", "+"))
}

#####
#
#           Install library FrF2
#
#####
install.packages("FrF2")
library(FrF2)
#####
#
#           Interaction plot
#
#####
IAPlot(g2)
#####
#
#           Reduced Model
#
#####

```

```
#####
g_reduced = lm(y_transformed ~ arm_pos + vert_pos)
summary(g_reduced)
summary.aov(g_reduced)
summary(g2)
```

Appendix - Python Codes

```
#####
#
#           First we enter the data
#
#####
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from scipy import stats

arm_pos = [-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,1,1,1,1,1,1,1,1,1,1]
vert_pos = [-1,-1,-1,-1,-1,-1,1,1,1,1,1,1,-1,-1,-1,-1,-1,1,1,1,1,1]
stop_ang = [-1,-1,-1,1,1,1,-1,-1,-1,1,1,1,-1,-1,-1,1,1,1,-1,-1,1,1]
y = [14,18,30,17,44,43,70,46,57,63,57,60,132,54,58,55,67,49,93,92,93,94,90,136]

df = pd.DataFrame({'arm_pos': arm_pos, 'vert_pos': vert_pos,
                  'stop_ang': stop_ang, 'y': y})
print(df)
#####
#
#           Now we fit the model
#
#####
Formula = 'y~arm_pos+vert_pos+stop_ang+arm_pos:vert_pos+arm_pos:stop_ang\
+vert_pos:stop_ang+arm_pos:vert_pos:stop_ang'
g = smf.ols(formula = Formula, data=df).fit()
#####
#
#           Residual vs Fitted Plot
#
#####
plt.figure()
plt.scatter(g.fittedvalues, g.resid, color='red', s=60)
plt.axhline(0, linestyle='--', color='black')
```

```

plt.xlabel('Fitted Values', fontsize=13)
plt.ylabel('Residuals', fontsize=13)
plt.title('Residuals vs Fitted')
plt.show()

#####
#
#                               QQ Plot
#
#####

plt.figure()
stats.probplot(g.resid, dist="norm", plot=plt)
plt.title('Normal Q-Q Plot')
plt.show()

#####
#
#                               Transform Data
#
#####
lambda1 = 0.3131548
y = np.array(y)
y_transformed = (y*lambda1 - 1) / lambda1
df['y_transformed'] = y_transformed
g2 = smf.ols("y_transformed ~ arm_pos + vert_pos + stop_ang + \
            arm_pos:vert_pos + arm_pos:stop_ang + \
            vert_pos:stop_ang + arm_pos:vert_pos:stop_ang", data=df).fit()

#####
#
#                               Residual vs Fitted Plot
#
#####

plt.figure()
plt.scatter(g2.fittedvalues, g2.resid, color='red', s=60)
plt.axhline(0, linestyle='--', color='black')
plt.xlabel('Fitted Values', fontsize=13)
plt.ylabel('Residuals', fontsize=13)
plt.title('Residuals vs Fitted')
plt.show()

#####
#
#                               QQ Plot
#
#####

plt.figure()
stats.probplot(g2.resid, dist="norm", plot=plt)
plt.title('Normal Q-Q Plot')
plt.show()

```

```

#####
#                                                                    #
#          Calculate the factorial effects                            #
#                                                                    #
#####
eff = 2 * g2.params[1:]
print(eff)
#####
#                                                                    #
#          Half Normal Plot Function                                #
#                                                                    #
#####
from scipy.stats import norm
def halfnormal(x):
    n = len(x)
    k = np.array(list(range(1, n+1)))
    halfn = 0.5 + 0.5*(k-.5)/n
    Xs = norm.ppf(halfn)
    Ys = np.sort(abs(x))
    plt.scatter(Xs,Ys)
    plt.xlabel("half-normal quantiles")
    plt.ylabel("absolute effects")
    plt.title("Half-Normal Plot")
#####
#                                                                    #
#          Plot half normal for eff                                  #
#                                                                    #
#####
halfnormal(eff)
#####
#                                                                    #
#          Main Effects Plots                                       #
#                                                                    #
#####
from statsmodels.graphics.factorplots import interaction_plot
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
factor_arrays = [np.array(arm_pos), np.array(vert_pos), np.array(stop_ang)]
factor_names = ['Arm Position', 'Vertical Position', 'Stop Angle']
y_arr = np.array(y_transformed)

for i, ax in enumerate(axes):
    interaction_plot(factor_arrays[i], np.ones(len(y)), y_arr,
                    ax=ax, colors=['red'], markers=['o'])
    ax.set_xlabel(factor_names[i])
    ax.set_ylabel('Mean Distance (in.)')
    ax.get_legend().remove()

```

```

plt.suptitle('Main Effects Plot')
plt.tight_layout()
plt.show()
#####
#                                                                 #
#                               Interaction Plots                  #
#                                                                 #
#####
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
pairs = [(0,1,'Arm Pos','Vert Pos'),
         (0,2,'Arm Pos','Stop Ang'),
         (1,2,'Vert Pos','Stop Ang')]

for ax, (i, j, name_i, name_j) in zip(axes, pairs):
    interaction_plot(factor_arrays[i], factor_arrays[j], y_arr,
                    ax=ax, colors=['red','black'], markers=['o','o'])
    ax.set_xlabel(name_i)
    ax.set_ylabel('Mean Distance (in.)')
    ax.set_title(f'{name_i} x {name_j}')
plt.suptitle('Interaction Effects Plot')
plt.tight_layout()
plt.show()
#####
#                                                                 #
#                               Reduced Model                        #
#                                                                 #
#####
g_reduced = smf.ols('y_transformed ~ arm_pos + vert_pos', data=df).fit()
print(g_reduced.summary())
print(g2.summary())

```